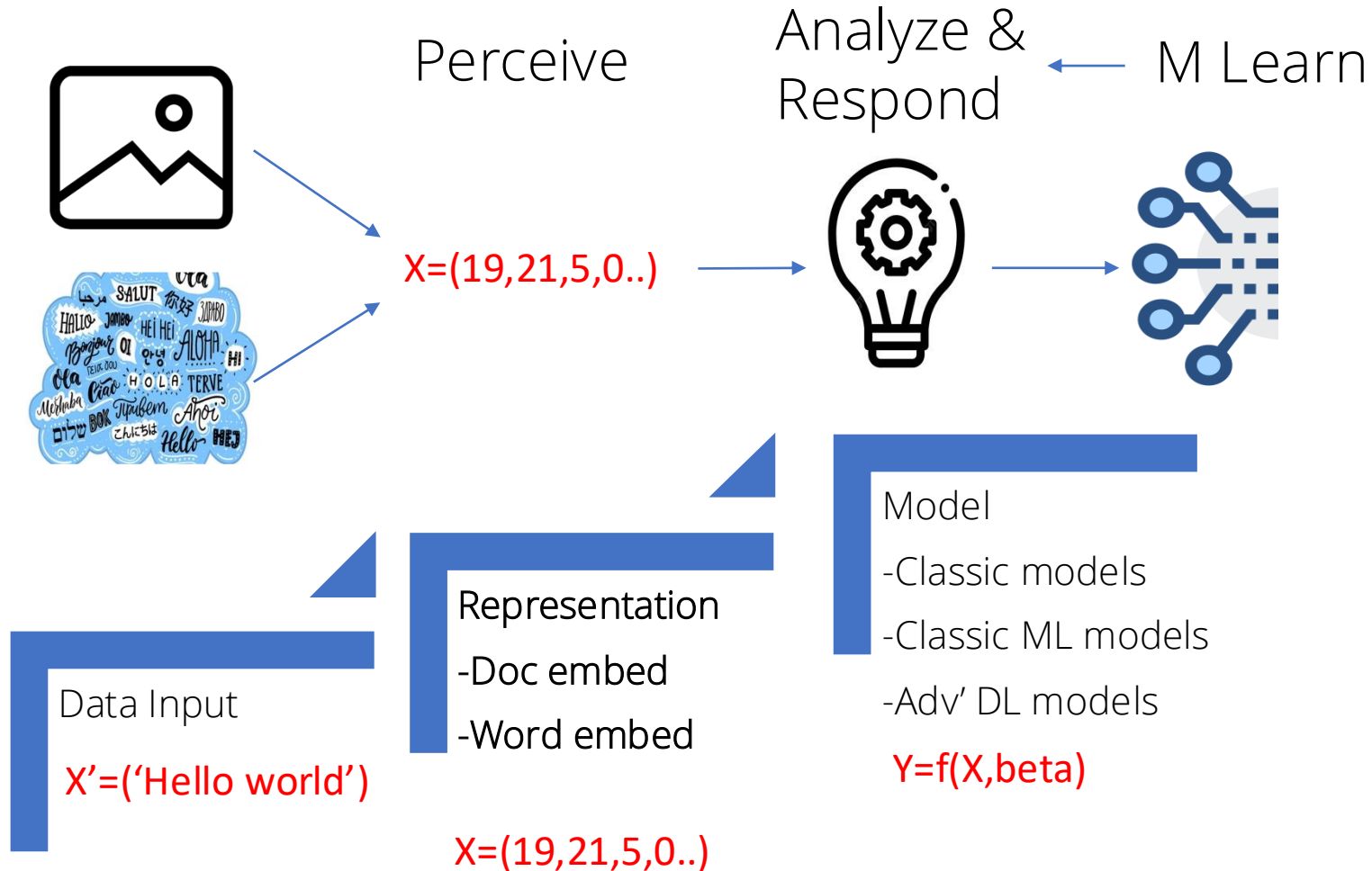


S4: Similarity and Clustering

Unstructured data analytics process



Similarity

- How similar are those documents?
- A query: look up the most similar sentence to the following document:
"Jinyang Zheng is a nice guy"

$$Q = \{q_1, q_2, \dots, q_i, \dots, q_n\} \quad i=1, 2, \dots, n$$

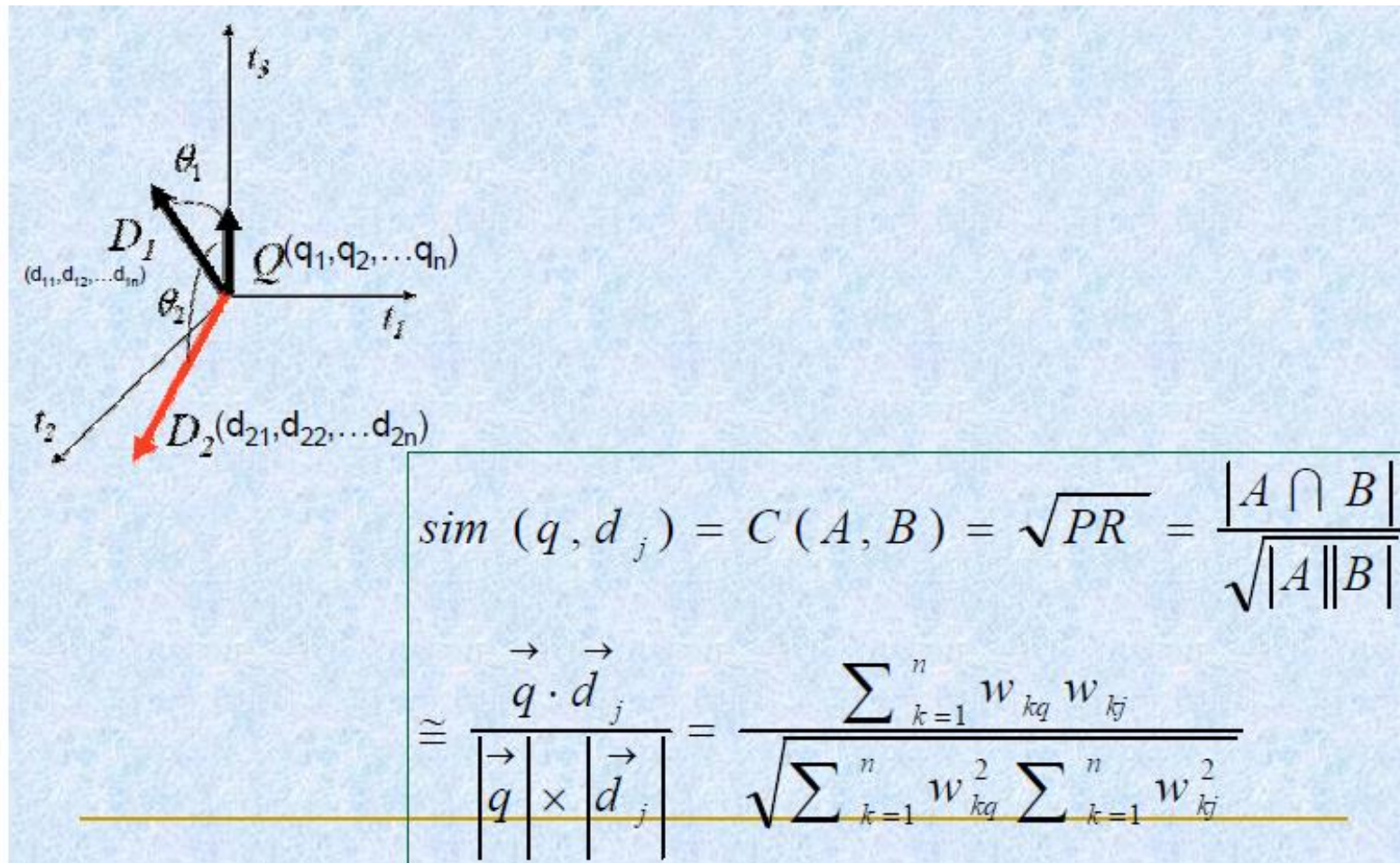
- "The instructor is a nice guy"
- "The TA is really bad"
- "I love my TA"
-

$$d_i = \{w_{i1}, w_{i2}, \dots, w_{ij}, \dots, w_{in}\} \quad i=1, 2, \dots, m, \quad j=1, 2, \dots, n$$

- If $\text{similarity}(Q, d_i) > \text{similarity}(Q, d_j)$,
We may get the result d_i is more relevant than d_j to Q

docs	<i>t1</i>	<i>t2</i>	<i>t3</i>	RSV=Q.Di
D1	1	0	1	4
D2	1	0	0	1
D3	0	1	1	5
D4	1	0	0	1
D5	1	1	1	6
D6	1	1	0	3
D7	0	1	0	2
D8	0	1	0	2
D9	0	0	1	3
D10	0	1	1	5
D11	1	0	1	3
<i>Q</i>	1	2	3	
	<i>q1</i>	<i>q2</i>	<i>q3</i>	

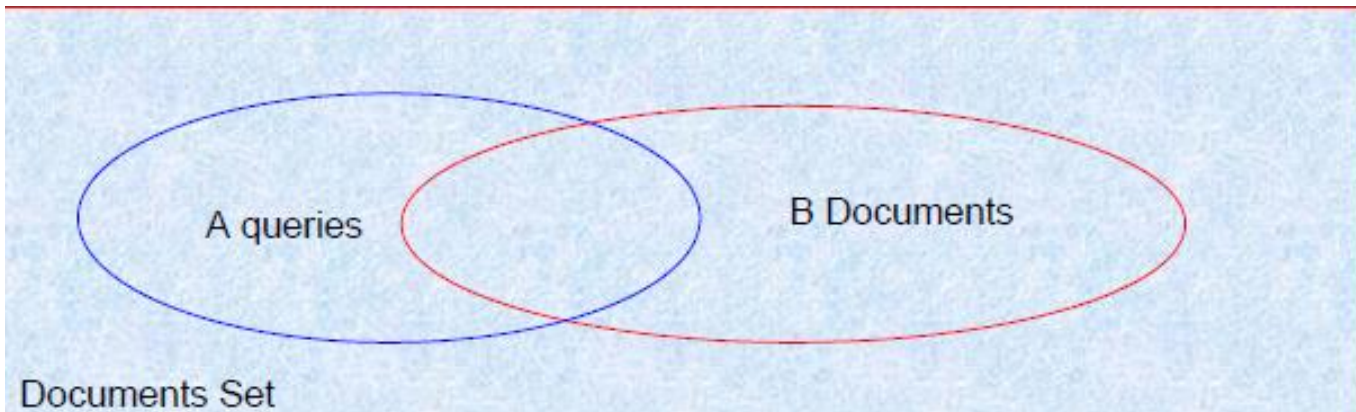
Cosine Coefficient



Jaccard Coefficient

$$\begin{aligned} \text{sim}(q, d_j) &= J(A, B) = \frac{|A \cap B|}{|A \cup B|} \\ &\approx \frac{\sum_{k=1}^n w_{kq} w_{kj}}{\sum_{k=1}^n w_{kq}^2 + \sum_{k=1}^n w_{kj}^2 - \sum_{k=1}^n w_{kq} w_{kj}} \end{aligned}$$

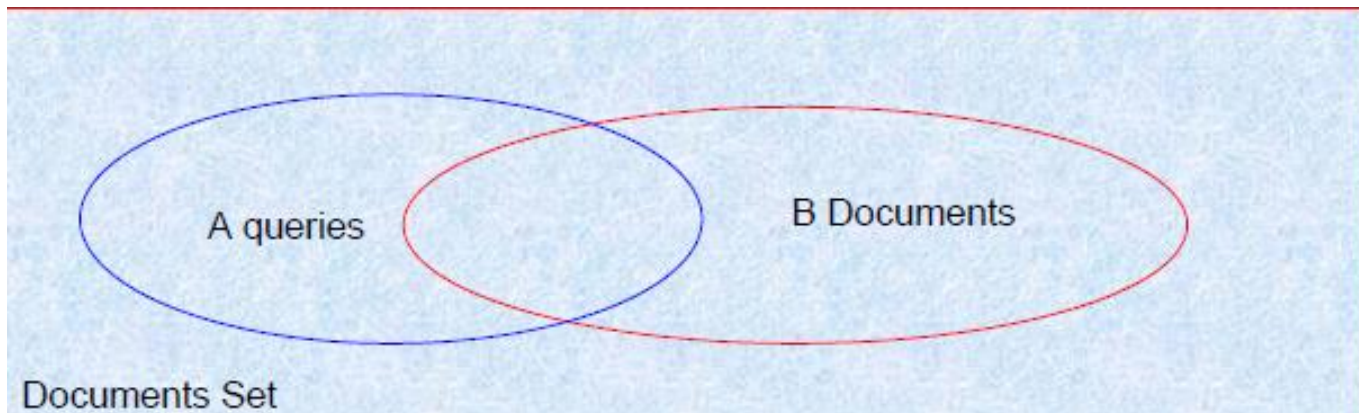
- $D1 = 2T1 + 3T2 + 5T3, (2, 3, 5)$
- $D2 = 3T1 + 7T2 + 1T3, (3, 7, 1)$
- $Q = 0T1 + 0T2 + 2T3, (0, 0, 2)$
- $J(D1, Q) = (2*0+3*0+2*5)/(2^2+3^2+5^2+2^2-2*0-3*0-2*5) = 10/32 = 0.31$
- $J(D2, Q) = (3*0+7*0+1*2)/(3^2+7^2+1^2+2^2-3*0-7*0-1*2) = 2/61 = 0.04$



Overlap Coefficient

$$\text{sim}(q, d_j) = O(A, B) = \frac{|A \cap B|}{\min(|A|, |B|)}$$

$$\approx \frac{\sum_{k=1}^n w_{kq} w_{kj}}{\min(\sum_{k=1}^n w_{kq}^2, \sum_{k=1}^n w_{kj}^2)}$$



Similarity measures between vectors

- A similarity measure is a function which computes **the degree of similarity** between a pair of vectors
- Assuming that Q and D are the sets of terms associated with a Query and Document:
 - Simple matching $\frac{|Q \cap D|}{|Q| + |D|}$
 - Dice Coefficient $\frac{|Q \cap D|}{|Q \cup D|}$
 - Jaccard Coefficient $\frac{|Q \cap D|}{|Q|^{1/2} \times |D|^{1/2}}$
 - Cosine Coefficient $\frac{|Q \cap D|}{\min(|Q|, |D|)}$
 - Overlap Coefficient
- Can be map to the range [-1, 1] or [0, 1], 0 or -1 shows minimum similarity, and 1 shows maximum similarity

Python: Similarity

```
#from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import *

vectorizer5 = CountVectorizer()
vectorizer5.fit(c3)
print(vectorizer5.vocabulary_)
v_c3 = vectorizer5.transform(c3).toarray()
print(v_c3)

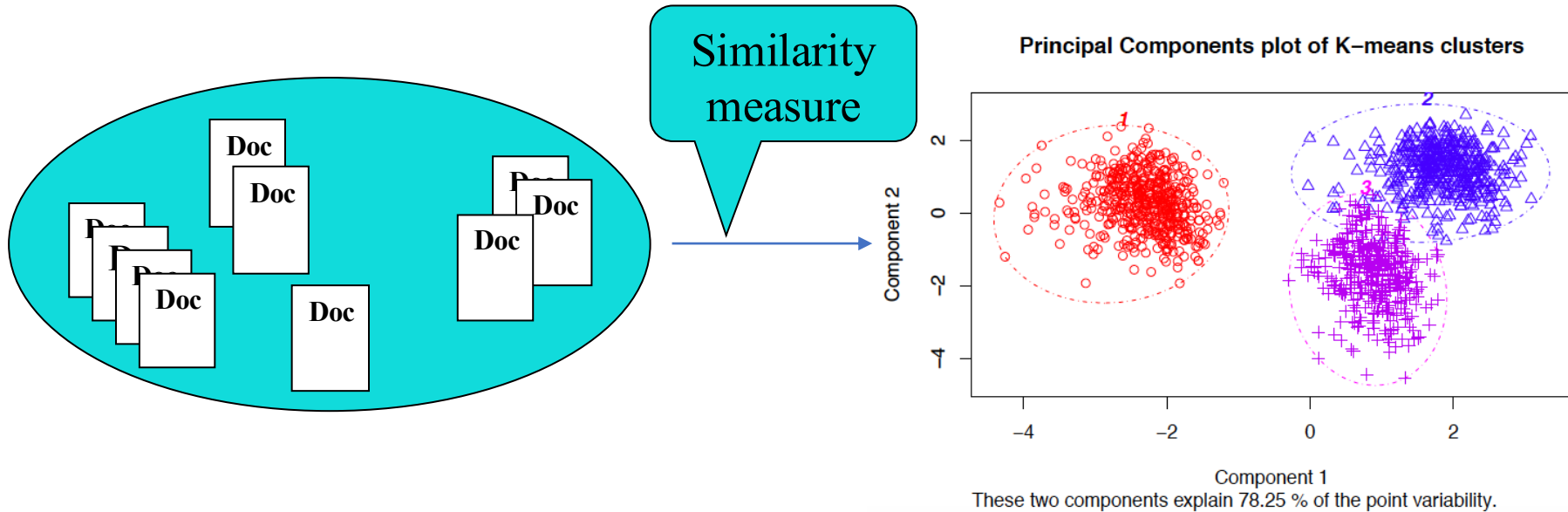
cosine_similarity([v_c3[0, :]], [v_c3[1, :]])
jaccard_similarity_score(v_c3[0, :], v_c3[1, :])
```


Limitation of classic similarity measure

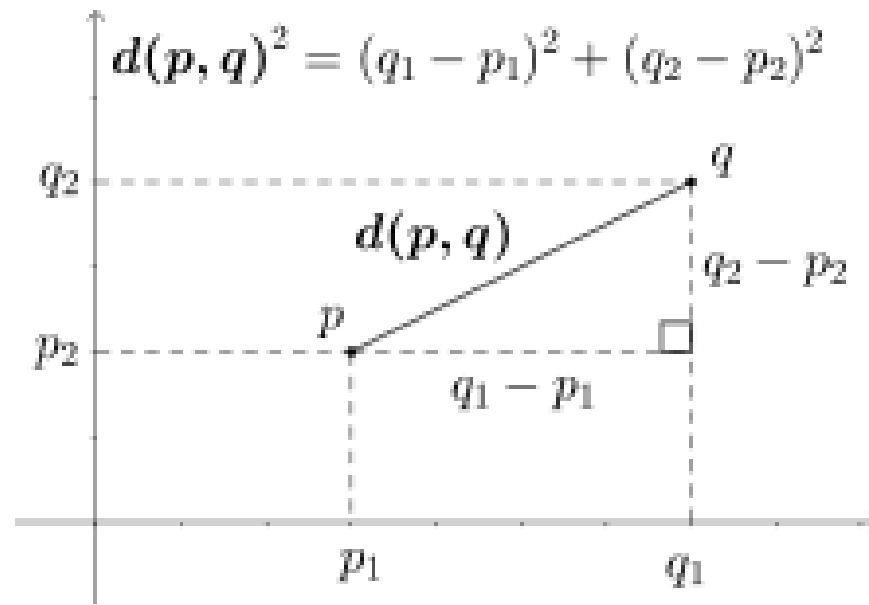
- Handling one query each time
- What if you want to compare **all the documents** and **categorize** them into **several groups**, in which documents are similar to each other?

Clustering

- **Given:** similarity measures between vectors (documents)
- **Find:** several clusters of vectors (documents) that are relevant to each other



The similarity measure here: euclidean distance

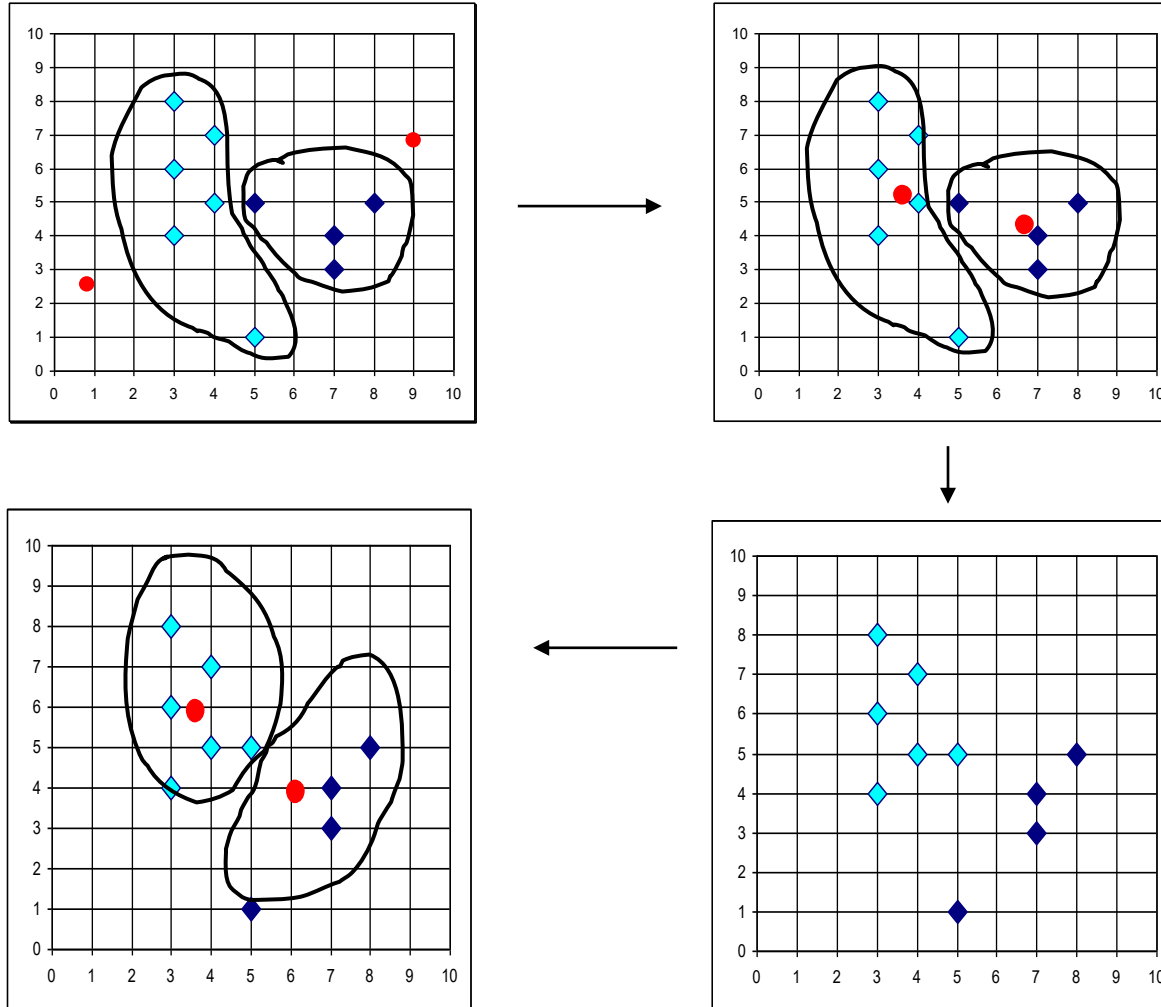


$$dis(q, d_j) = d_E(q, d_j) = \sqrt{\sum_{k=1}^n (w_{kq} - w_{kj})^2}$$

Partitioning algorithms

- **Partitioning method:** construct a partition of n documents into a set of k clusters
- **Given:** a set of (n) documents and the number k
- **Find:** a partition of k clusters that optimizes the chosen partitioning criterion
- **Global optimal:** exhaustively enumerate all partitions
- **Heuristic methods:** k-means algorithm
- k-means: each cluster is represented by the center of the cluster
 - Step 1: Initialize/update K (random) means
 - Step 2: Assign each dot to the nearest mean and form/update K clusters

K-mean



Clustering Result

- “The instructor is a nice guy”
- “The TA is really bad”
- “I love my TA”
-

	t1	t2	t3	t4	...
d1	0	0.12	0.61	0.76	...
d2	0.21	0.55	0.01	0	...
d3	0.44	0.12	0.19	0.02	...
...	...	...	...	...	...

	G1	G2
d1	0	1
d2	1	0
d3	1	0
...	...	...

	t1	t2	t3	t4	...
G1	0.07	0.09	0.55	0.81	...
G2	0.33	0.25	0.11	0.01	...

Python: K-mean

```
d10 = "I am a data scientist"
c4 = [d1,d2,d3,d4,d5,d6,d7,d8,d9,d10]

vectorizer6 = TfidfVectorizer(stop_words='english')
X = vectorizer6.fit_transform(c4)

modell1 = KMeans(n_clusters=2)
modell1.fit(X)
modell1.cluster_centers_
modell1.fit_transform(X)
print( modell1.labels_.tolist())

order_centroids = modell1.cluster_centers_.argsort()[:, :-1]
terms = vectorizer6.get_feature_names()
for ind in order_centroids[1, :4]:
    print(terms[ind])
for ind in order_centroids[0, :4]:
    print(terms[ind])
```

Hierarchical clustering

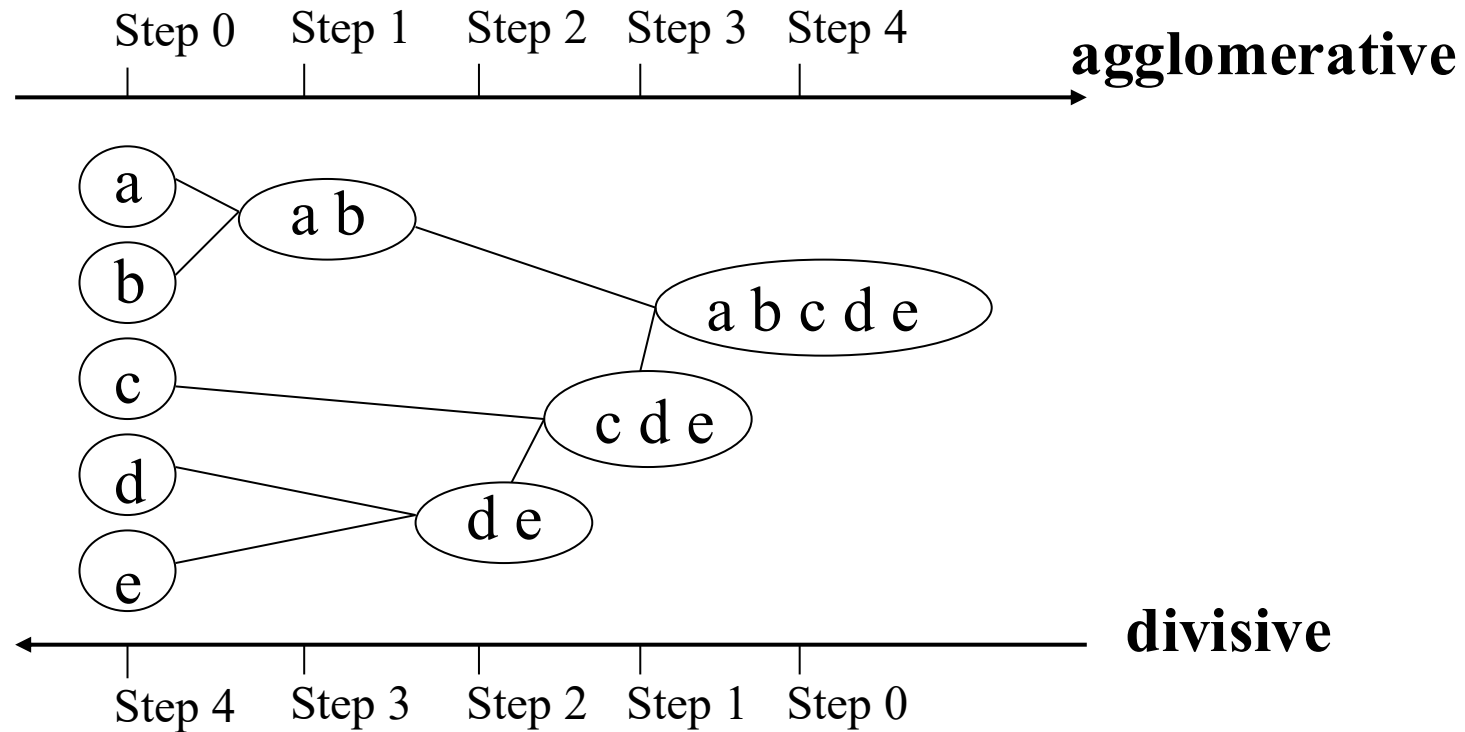
- **Agglomerative:**

- Start with each document being a single cluster.
- Eventually all document belong to the same cluster.
- In each step, merge the two closest two clusters
- Objective function: **minimizes the sum of squared distances within all clusters.**

- **Divisive:**

- Start with all document belong to the same cluster.
- Eventually each node forms a cluster on its own.
- In each step, separate the two farthest two potential clusters
- **Note:** Needs a termination condition
 - The final mode in both Agglomerative and Divisive is of no use.

Agglomerative and divisive



Python: Agglomerative Clustering

```
##Hierarchical cluster
from sklearn.cluster import AgglomerativeClustering

model2 = AgglomerativeClustering(n_clusters=2,
affinity='euclidean', linkage='ward')
model2.fit_predict(X.toarray())
print(model2.labels_)
```

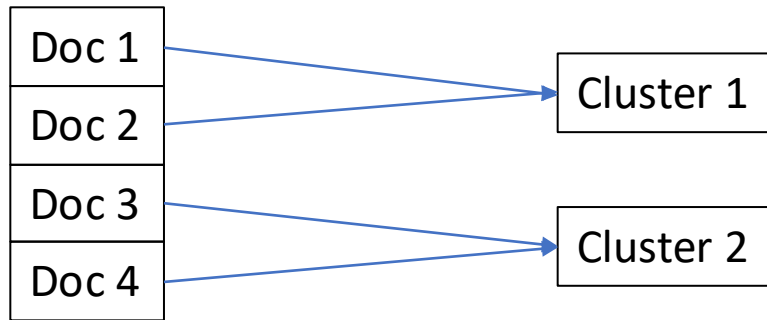

Min Group Project 1b

- Use the same data as in 1a, transform those reviews into a term-document matrix, lemmatize all the words, remove the stop-words and punctuations, set the minimal document frequency for each term to be 5 and include unigram and bi-gram.
- Use k-mean model to extract the cluster of each document assuming there are 6 clusters.
- Report the cluster of the first 10 restaurant reviews (id = [1:10]) and the first 10 movie reviews (id = [501:510]).
- Find the top-5 terms (terms with the top-5 highest weights) for each of the 6 clusters. Based on those terms, describe what those topics are about.

Submit the code and the outputs of the last two steps.

Limitation of clustering

- Fundamental assumption: each doc belongs to one cluster, many to one relationship



- What if a doc might belong to multiple clusters?
 - E.g., can a movie (review) be about both of action and horror?
- Question: any generalization of clustering wherein a doc belongs to multiple clusters?

Idea of Latent Dirichlet Allocation

- Generalization of clustering
- A **dot** belongs to
- one **cluster** in conventional clustering
- **now in LDA, multiple clusters**
- **Probabilistically (weighted)**

LDA Result

$$M = K * W$$

- “The instructor is a nice guy”
- “The TA is really bad”
- “I love my TA”
-

	t1	t2	t3	t4	...
d1	0	0.12	0.61	0.76	...
d2	0.21	0.55	0.01	0	...
d3	0.44	0.12	0.19	0.02	...
...	...	...	...	...	...

	G1	G2
d1	0.23	0.77
d2	0.66	0.34
d3	0.81	0.19
...	...	...

	t1	t2	t3	t4	...
G1	0.07	0.09	0.55	0.81	...
G2	0.33	0.25	0.11	0.01	...

	G1	G2
d1	0	1
d2	1	0
d3	1	0
...	...	...

- Ref: [Latent Dirichlet Allocation by Blei, Ng and Jordan](#)
- Documents are produced from a mixture of topics (probabilistically)
- Topics described by the words of highest weights

Latent Dirichlet Allocation (LDA)

- General idea: perform a low-rank matrix factorization
- **$M = KW$**
 - **M** : $M(\text{term } w \mid \text{document } d)$ term-document matrix
 - **K** : $P(\text{topic } t \mid \text{document } d)$: each document's topic weight distribution
 - **W** : $W(\text{term } w \mid \text{topic } t)$: each topic's term weights (centroid)

	W1	W2	W3	<u>Wn</u>
D1	0	2	1	3
D2	1	4	0	0
D3	0	2	3	1
<u>Dn</u>	1	1	3	0

 $=$

	K1	K2	K3	K
D1	1	0	0	1
D2	1	1	0	0
D3	1	0	0	1
<u>Dn</u>	1	0	1	0

 $\times$

	W1	W2	W3	<u>Wm</u>
K1	0	1	1	1
K2	1	1	1	0
K3	1	0	0	1
K	1	1	0	0

LDA Inference: Collapsed Gibbs Sampling

- Step 1: Go through each **document** and randomly assign each **term** in the document to one of **K topics** (K is chosen beforehand)
- Step 2: For each document d , go through each term w and compute:
 - $P(\text{topic } t \mid \text{document } d)$: proportion of terms in document d that are assigned to topic t
 - $P(\text{term } w \mid \text{topic } t)$: proportion of assignments to topic t , over all documents d , that come from term w
- Step 3: Reassign term w a new topic t' , where we choose topic t' with probability:
 - A function of $P(\text{term } w \mid \text{topic } t)$, $P(\text{topic } t' \mid \text{doc } d)$, $P(\text{term } w \mid \text{doc } d)$
 - $= P(\text{topic } t' \mid \text{term } w, \text{doc } d)$
- Back to Step 2, until we reach a steady state where topic assignments are pretty good.

LDA topic modeling

- **One dot** belongs to (the mixture of) **multiple clusters**
- **One cluster** corresponds to a specific vector in the feature vector space (i.e., **the center**)
- **One document** is related to (a mixture of) **multiple topics**
- **One topic** can be described by **a feature vector** which tells the weight for each word.
 - The topic can also be described by multiple high-weight words (example topics: "sports", "medicine", "movies", "finance")

Python: LDA topic modeling

```
vectorizer7 = CountVectorizer(stop_words='english')
X2 = vectorizer7.fit_transform(c4)
terms = vectorizer7.get_feature_names()

from sklearn.decomposition import LatentDirichletAllocation
lda = LatentDirichletAllocation(n_topics=4).fit(X2)

for topic_idx, topic in enumerate(lda.components_):
    print("Topic %d:" % (topic_idx))
    print(" ".join([terms[i] for i in topic.argsort()[:-4:-1]]))

lda.transform(X2[1])
print(lda.components_)
```

Python: Using gensim

```
import gensim

dictionary = gensim.corpora.Dictionary(processed_c4)
bow_c4 = [dictionary.doc2bow(doc) for doc in processed_c4]
print(bow_c4[1])

lda_model = gensim.models.LdaModel(bow_c4, num_topics=5,
id2word=dictionary)

for idx, topic in lda_model.print_topics():
    print('Topic: {} \nWords: {}'.format(idx, topic))

lda_model[bow_c4[1]]
```

Min Group Project 1b cont'

- Use the same representation as in 1b.
- Use LDA model to extract the topic distribution of each document assuming there are 6 topics.
- Report the topic distribution of the first 10 restaurant reviews (id = [1:10]) and the first 10 movie reviews (id = [501:510]).
- Find the top-5 terms (terms with the top-5 highest weights) for each of the 6 topics. Based on those terms, describe what those topics are about.

Submit the code and the outputs of the last two steps.

